BÁO CÁO THỰC HÀNH

LAB-02 | Device driver và quản lý file system

Character device • procfs/sysfs • NAND/MTD/JFFS2 • BusyBox init

ARMv7-A • QEMU vexpress-a9 • Linux 5.15

DANH SÁCH NHÓM THỰC HÀNH

Họ và tên	MSSV
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Đơn vị thực hiện: Nhóm 5 sinh viên • Môn: Embedded Linux Systems

Tổng hợp dữ liệu thực hành: 09–10/2026

Tất cả hình minh chứng terminal sử dụng từ bài thực hành của nhóm; sơ đồ được thiết kế lại cho báo cáo này.

TỔNG QUAN & PHƯƠNG PHÁP KIỂM TRA

Lab 2 mở rộng từ hệ thống tối giản ở Lab 1. Nhóm viết character device driver dùng file_operations, xây dựng hai kênh quan sát procfs/sysfs và kiểm tra hệ thống lưu trữ raw NAND giả lập bằng MTD/JFFS2. Phần driver chạy trong QEMU ARMv7-A Linux 5.15; phần nandsim/JFFS2 chạy trên Ubuntu HOST, không phải trong target.

Khâu	Đầu vào thực tế	Kết quả đối chiếu
A · Device driver	lab2_driver.ko, kernel target 5.15.0	ARM ELF32, major 240, read/write đúng
B · procfs/sysfs	/proc/lab2_info và 3 file sysfs	test_procfs.sh báo PASS
C · NAND/JFFS2	Ubuntu host, nandsim /dev/mtd0	16 MiB; erase 16 KiB; mount/remount đúng
D · Init tự động	rcS, initramfs_lab2.cpio.gz	Module load và tạo /dev/lab2 khi boot

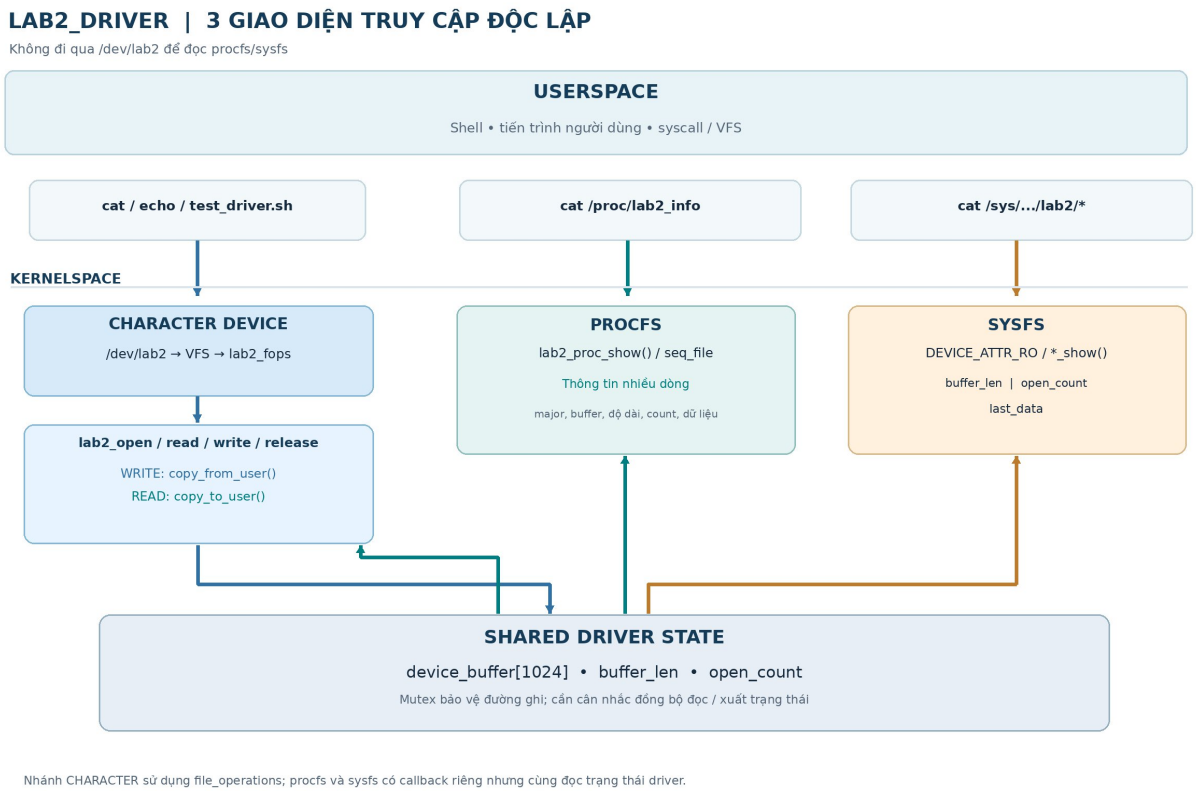
1. MÔI TRƯỜNG VÀ MỤC TIÊU

Môi trường	Giá trị sử dụng
HOST	Ubuntu x86_64; kernel 6.8.0-147-generic (theo log nhóm).
TARGET	Cortex-A9 ARMv7-A; QEMU vexpress-a9; -m 512M, -smp 2 theo phiên test.
Cross compiler	arm-linux-gnueabihf-; Linux kernel target 5.15.0.
Driver	lab2_driver.ko; register_chrdev, character device /dev/lab2.
Flash giả lập	nandsim /dev/mtd0; 16 MiB; 1024 eraseblocks × 16 KiB.
Filesystem	JFFS2 với eraseblock 0x4000, mounted /mnt/jffs2 trên HOST.

Không trộn kết quả Tài liệu PDF tham khảo dùng /dev/mtd1 và NAND 32 MiB trên máy khác; nhóm này quan sát /dev/mtd0 dung lượng 16 MiB. Tất cả bảng và kết luận dưới đây dùng dữ liệu của nhóm.

2. SƠ ĐỒ KIẾN TRÚC VÀ LUỒNG DỮ LIỆU DRIVER

Cùng một kernel module nhưng userspace nhìn thấy ba cửa vào khác nhau: character node để truyền dữ liệu, procfs để đọc báo cáo thống kê, và sysfs để truy vấn từng thuộc tính. Chúng sử dụng callback khác nhau và cùng quan sát các biến trạng thái của driver. Không thể vẽ procfs hay sysfs như bước “đi qua /dev/lab2”.



Hình 1. Sơ đồ kiến trúc vẽ lại: các đường xử lý độc lập, callback và trạng thái dùng chung.

2.1. Nhánh character device — luồng WRITE / READ

```
WRITE: echo/caller → write() → VFS → lab2_write()
    → copy_from_user() → device_buffer → buffer_len
READ : cat/caller → read() → VFS → lab2_read()
    → copy_to_user() → bộ đệm ứng dụng
```

Node /dev/lab2 mang major 240, minor 0 trong log lần chạy. Khi user gọi read/write, VFS định tuyến vào các hàm mà driver công bố qua file_operations (lab2_open, lab2_read, lab2_write, lab2_release). Dữ liệu thực được lưu trong device_buffer[1024]; buffer_len biểu thị lượng dữ liệu có hiệu lực.

2.2. Nhánh procfs và sysfs

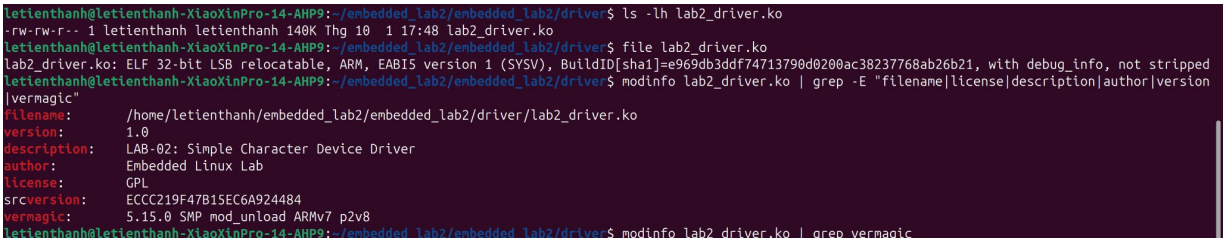
/proc/lab2_info dùng seq_file để trả về một bản thống kê nhiều trường như major, buffer size, data length, open count và last data. Trong sysfs, ba thuộc tính read-only /sys/class/lab2_class/lab2/{buffer_len, open_count, last_data} cung cấp từng trường riêng qua callback *_show(). Các file dev/uevent khác thuộc hạ tầng thiết bị của kernel, không phải ba thuộc tính custom của lab.

Trong thao tác ghi, mutex ngăn hai lệnh write can thiệp đồng thời vào vùng dữ liệu đã khóa. Tuy nhiên, muốn bảo đảm snapshot nhất quán khi read và các callback *_show() chạy cạnh tranh với write, cần kiểm tra hoặc bổ sung đồng bộ ở đường đọc. Bài test tuần tự chưa chứng minh stress concurrency.

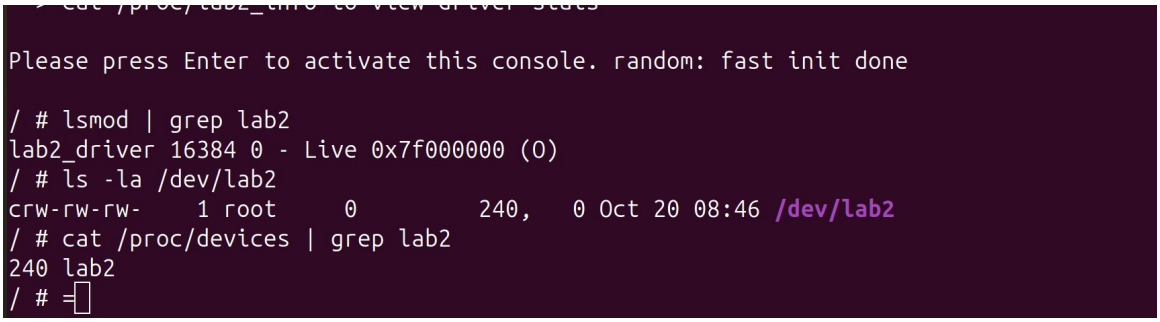
3. PHẦN A — MODULE VÀ CHARACTER DEVICE

3.1. Kiểm tra ELF, vermagic và đăng ký thiết bị

Module được build dạng ELF32 ARM EABI5 và có vermagic “5.15.0 SMP mod_unload ARMv7 p2v8”. Thông số này đối chiếu với kernel target. Khi nạp thành công, log hiển thị lab2_driver: loaded, major=240; lsmod ghi nhận module đang live, còn /proc/devices và ls -l /dev/lab2 xác thực mapping major/minor.



Hình 2. file và modinfo của lab2_driver.ko (nguồn: log nhóm).



Hình 3. Module live; character node major 240/minor 0 trên target.

3.2. Đọc/ghi dữ liệu và dmesg

Nhóm ghi chuỗi “Hello from userspace, LAB-02!” vào /dev/lab2 rồi đọc lại nguyên vẹn; script tiếp tục kiểm tra Message_1, Message_2 và Message_3. Các log ghi received N bytes và sent N bytes cho phép đối chiếu hàm write/read; kết quả test_driver.sh in “Test PASSED”. Byte newline do echo thêm vào cũng được tính trong kích thước dữ liệu.



Hình 4. Chu kỳ read/write lặp lại và trạng thái Test PASSED.

4. PHẦN B — QUAN SÁT TRẠNG THÁI QUA PROCFS/SYSFS

Script test_procfs.sh trước tiên đọc thống kê trước ghi, sau đó đẩy chuỗi “Embedded Linux Lab2 Test Data” qua character node rồi đọc lại procfs và sysfs. Trên log của nhóm, chiều dài dữ liệu đạt 30 byte, open_count tăng và last_data tương ứng với chuỗi được ghi. Những số count có thể thay đổi sau mỗi lần kiểm tra vì open_count đếm số lần mở kể từ khi module được nạp.

```
/ # echo "===== RUN TEST_PROCFS.SH ====="
===== RUN TEST_PROCFS.SH =====
/ # test_procfs.sh
=== Test procfs & sysfs ===
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 0 bytes
Open count : 0
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 1
Last data : [Embedded Linux Lab2 Test Data
]
```

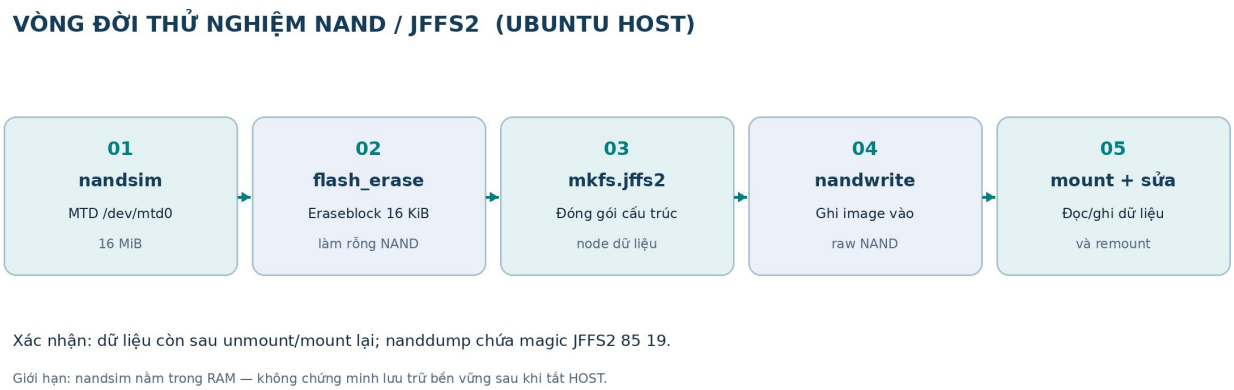
Hình 5. Thống kê /proc/lab2_info trước và sau WRITE.

```
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 18
Last data : [Embedded Linux Lab2 Test Data
]
/ # echo "===== SYSFS ATTRIBUTES ====="
===== SYSFS ATTRIBUTES =====
/ #
/ # SYSFS=/sys/class/lab2_class/lab2
/ #
/ # ls -la $SYSFS
lrwxrwxrwx    1 root     0               0 Oct 20 08:51 /sys/class/lab2_class/lab2 -> ../../devices/virtual/lab2_class/lab2
/ #
/ # echo "buffer_len = $(cat $SYSFS/buffer_len)"
buffer_len = 30
/ # echo "open_count = $(cat $SYSFS/open_count)"
open_count = 18
/ # echo "last_data  = $(cat $SYSFS/last_data)"
last_data  = Embedded Linux Lab2 Test Data
```

Hình 6. Đối chiếu buffer_len/open_count/last_data tại sysfs.

Mỗi file sysfs phục vụ đúng một loại dữ liệu trạng thái; đây là giao diện thuận tiện cho script giám sát. procfs phù hợp hơn khi cần trình bày nhiều trường cho người đọc. Trong thiết kế sản phẩm, lựa chọn giao diện phụ thuộc ABI mong muốn, không phải vì procfs là một bước của lệnh write().

5. PHẦN C — MÔ PHỎNG NAND VÀ JFFS2 TRÊN HOST



Xác nhận: dữ liệu còn sau unmount/mount lại; nanddump chứa magic JFFS2 85 19.

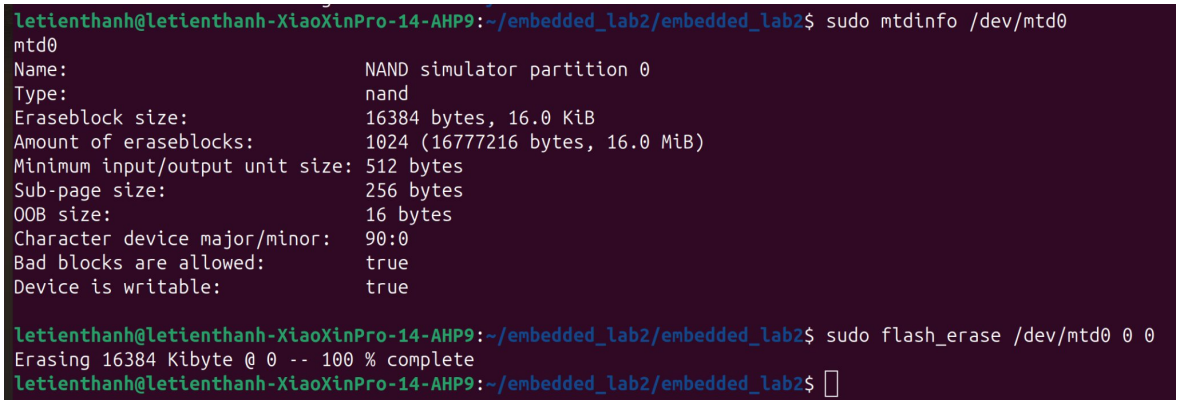
Giới hạn: nandsim nằm trong RAM — không chứng minh lưu trữ bền vững sau khi tắt HOST.

Hình 7. Chuỗi thao tác được thực hiện trên Ubuntu HOST, không chạy trong QEMU.

5.1. Tạo MTD và xác minh chip trước khi xóa

Nhóm dùng nandsim với first_id_byte=0x20 và second_id_byte=0x33. /proc/mtd ghi “NAND simulator partition 0” tại /dev/mtd0; dung lượng 0x01000000 (16 MiB), erase size 0x00004000 (16 KiB). mtdinfo cho 1.024 eraseblocks, page 512 B và OOB 16 B. Đây là thông số của lần thử, không suy ra từ ví dụ 32 MiB trong tài liệu khác.

```
cat /proc/mtd
sudo mtdinfo /dev/mtd0
# Chỉ erase sau khi đã xác nhận đúng NAND simulator
```



Hình 8. Kích thước NAND và kết quả flash_erase trên host.

An toàn thao tác MTD Không chạy flash_erase/nandwrite dựa duy nhất vào tên mtd0: thứ tự MTD có thể khác giữa các host. Luôn kiểm tra /proc/mtd và mtdinfo để tránh thao tác nhầm lên thiết bị flash thật.

5.2. Tạo image, ghi và mount

Bộ dữ liệu thí nghiệm gồm jffs2_root/etc/lab2.txt và jffs2_root/data/test.txt. mkfs.jffs2 dùng eraseblock=0x4000, tức khớp kích thước eraseblock của nandsim. Image JFFS2 sau khi tạo được nandwrite vào thiết bị MTD rồi mount bằng mtdblock. Khi mount thành công, nhóm đọc được cả file cấu hình và dữ liệu mẫu.

```
mkfs.jffs2 -d jffs2_root -o rootfs.jffs2 -e 0x4000 -n
sudo nandwrite -p /dev/mtd0 rootfs.jffs2
sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
```



Hình 9. mkfs.jffs2 và nandwrite ghi image lên nandsim.

```
Try: sudo apt install <deb name>
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo modprobe mtdblock
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ ls -l /dev/mtdblock*
brw-rw---- 1 root disk 31, 0 Thg 10 20 16:02 /dev/mtdblock0
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ find /mnt/jffs2 -type f -o -type d
/mnt/jffs2
/mnt/jffs2/etc
/mnt/jffs2/etc/lab2.txt
/mnt/jffs2/data
/mnt/jffs2/data/test.txt
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/etc/lab2.txt
Embedded Linux Lab 2 - JFFS2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 10. Mount JFFS2 và đọc lại hai tệp dữ liệu.

5.3. Remount, raw dump và giới hạn persistence

Nhóm ghi nối vào data/test.txt và tạo runtime.txt trên filesystem đã mount, đồng bộ dữ liệu, unmount và mount lại. Kết quả giữ nguyên nội dung sau remount. nanddump không ghi nhận bad block hay ECC failed trong đoạn dữ liệu kiểm tra; hexdump bắt đầu bằng 85 19 (magic 0x1985 ở little-endian), phù hợp với định dạng JFFS2.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Persistent after remount\n" >> /mnt/jffs2/data/test.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Created during runtime\n" > /mnt/jffs2/runtime.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sync
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo umount /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 11. Thử nghiệm sửa dữ liệu và remount, các file vẫn đọc được.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd0
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ hexdump -C /tmp/dump.bin | head -20
00000000  85 19 01 e0 2c 00 00 00  5f 56 f1 e0 01 00 00 00  |....,..._V......|
00000010  00 00 00 00 02 00 00 00  93 2d d7 6a 04 04 00 00  |.........-.j....|
00000020  67 32 eb d2 7f 2c b7 8c  64 61 74 61 85 19 02 e0  |g2...,..data....|
00000030  44 00 00 00 1d fb f7 98  02 00 00 00 01 00 00 00  |D...............|
00000040  fd 41 00 00 e8 03 e8 03  00 00 00 00 93 2d d7 6a  |.A...........-.j|
00000050  93 2d d7 6a 93 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 47 7c 5a d2  |............G|Z.|
00000070  85 19 01 e0 2b 00 00 00  e6 6e 26 7d 01 00 00 00  |....+....n&}....|
00000080  01 00 00 00 03 00 00 00  db 2d d7 6a 03 04 00 00  |.........-.j....|
00000090  78 0e 89 80 db 85 f4 d1  65 74 63 ff 85 19 02 e0  |x.......etc.....|
000000a0  44 00 00 00 1d fb f7 98  03 00 00 00 01 00 00 00  |D...............|
000000b0  fd 41 00 00 e8 03 e8 03  00 00 00 00 db 2d d7 6a  |.A...........-.j|
000000c0  db 2d d7 6a db 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
000000d0  00 00 00 00 00 00 00 00  00 00 00 00 9c 8f c9 d3  |................|
000000e0  85 19 01 e0 30 00 00 00  78 be 3e fa 02 00 00 00  |....0...x.>.....|
000000f0  02 00 00 00 04 00 00 00  93 2d d7 6a 08 08 00 00  |.........-.j....|
00000100  f2 b9 13 56 12 9b eb d3  74 65 73 74 2e 74 78 74  |...V....test.txt|
00000110  85 19 02 e0 59 00 00 00  5f 74 84 3a 04 00 00 00  |....Y..._t.:....|
00000120  01 00 00 00 b4 81 00 00  e8 03 e8 03 15 00 00 00  |................|
00000130  93 2d d7 6a 93 2d d7 6a  93 2d d7 6a 00 00 00 00  |.-.j.-.j.-.j....|
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 12. Dữ liệu raw NAND: JFFS2 magic 85 19.

Giới hạn kiểm thử nandsim lưu nội dung trong RAM của host. Bài test chứng minh tính nhất quán qua unmount/mount lại khi nandsim vẫn sống, chứ chưa kiểm thử độ bền dữ liệu qua tắt máy hoặc độ bền của NAND vật lý.

6. PHẦN D — TÍCH HỢP DRIVER VÀO BOOT BUSYBOX

Với initramfs đã bổ sung lab2_driver.ko và hai shell script, BusyBox init đọc /etc/inittab, chạy rcS để mount các pseudo-filesystem, nạp module qua insmod và tạo node /dev/lab2. Khi đưa hai script vào hệ thống đã auto-load, cần tránh gọi insmod lặp gây lỗi “File exists”; nhóm đã kiểm tra trạng thái module trước khi thử nạp thêm.

```
Kernel → /sbin/init → /etc/inittab
    → ::sysinit:/etc/init.d/rcS
    → insmod /lib/modules/5.15.0/lab2_driver.ko
    → /dev/lab2 (major 240 / minor 0) → shell
```



Hình 13. Kernel boot/rcS nạp driver và khởi tạo /dev/lab2.



Hình 14. Đối chiếu module và script bên trong archive initramfs.

Lệnh kiểm thử sau boot: lsmod, ls -l /dev/lab2, cat /proc/lab2_info, đọc ba thuộc tính sysfs, và chạy test_driver.sh/test_procfs.sh. Các ảnh và log trong Phần A/B xác nhận hai script hoàn tất thành công. Đây là kiểm chứng tích hợp driver vào môi trường khởi động ARM, độc lập với bước mô phỏng NAND trên HOST.

7. FILESYSTEM — CHỌN CÔNG NGHỆ ĐÚNG NỀN TẢNG

Tiêu chí	ext4	JFFS2	cramfs	UBIFS	SquashFS
Ghi dữ liệu	Có	Có	Không	Có	Không
Lớp lưu trữ	Block	Raw MTD	ROM / image	UBI trên MTD	Image chỉ đọc
Wear leveling	FTL bên dưới	GC/flash-aware	Không	UBI	Không
Nén	Không mặc định	Có	Có	Có	Có
Đặc trưng mount	Journal	Quét node flash	Nhanh	Index trên flash	Đọc nhanh
Phù hợp	eMMC/SD	Flash nhỏ	Firmware RO	NAND lớn	Rootfs bất biến

7.1. Bài toán thiết bị IoT có NAND 32 MB và OTA

Nếu yêu cầu cập nhật từ xa, kiến trúc phù hợp cần định nghĩa được chiến lược cập nhật và phục hồi khi mất điện. Có thể phân chia vùng firmware A/B, cập nhật image ở vùng không chạy và kiểm tra trước khi đổi phiên boot; dữ liệu biến đổi lưu tại phân vùng writable. Với NAND raw, UBI/UBIFS là một lựa chọn nếu tài nguyên đáp ứng, còn JFFS2 có thể hợp lý cho phân vùng nhỏ. SquashFS phù hợp cho firmware chỉ đọc khi có lớp lưu trữ tương thích. Đây là đề xuất thiết kế, chưa được thử nghiệm OTA trong Lab 2.

8. NĂM CÂU HỎI PHÂN TÍCH

8.1. Vì sao không dùng memcpy với user pointer?

Bộ đệm userspace tồn tại trong không gian địa chỉ ảo của tiến trình gọi; con trỏ có thể không hợp lệ hoặc gặp page fault. API copy_from_user()/copy_to_user() dùng đường truy cập uaccess và cơ chế xử lý fault được kernel hỗ trợ, trả

số byte chưa sao chép. Driver nên chuyển lỗi truy cập sang -EFAULT theo hợp đồng của API. memcpy không mang sẵn cơ chế xử lý user-pointer này.

8.2. Mutex giải quyết race condition nào?

device_buffer và buffer_len là dữ liệu dùng chung. Nếu hai tiến trình ghi đồng thời, nội dung và độ dài cuối cùng có thể không nhất quán. Mutex tuần tự hóa vùng ghi chứa copy/cập nhật dữ liệu. Vì copy_from_user có khả năng sleep, mutex phù hợp hơn spinlock cho tình huống này. Kiểm thử hiện có chủ yếu tuần tự, chưa chứng minh đầy đủ tính nhất quán của đường read và show khi có nhiều luồng.

8.3. procfs khác sysfs?

procfs trong bài cho một bản mô tả nhiều trường bằng seq_file; sysfs thuộc driver model, ánh xạ từng trạng thái qua file thuộc tính của device kobject. Nhờ vậy userspace dễ đọc từng trường buffer_len/open_count/last_data. Hai đường đọc độc lập với character file_operations nhưng cùng quan sát dữ liệu của driver.

8.4. Tại sao JFFS2 cần eraseblock?

Cell NAND thường chỉ lập trình theo hướng bit 1→0; muốn phục hồi trạng thái 1 phải erase cả eraseblock. JFFS2 ghi thay đổi thành node mới, đánh dấu node cũ không còn hiệu lực và dùng garbage collection để thu hồi vùng flash. Đó là lý do eraseblock của mkfs.jffs2 cần tương thích thông số MTD và không thể áp trực tiếp cách ghi của ext4 trên thiết bị block vào raw NAND.

8.5. Nếu insmod trong rcS thất bại?

BusyBox init không tự sinh kernel panic chỉ vì một lệnh trong rcS trả mã lỗi. Kịch bản còn phụ thuộc cách script xử lý status, set -e hay exit. Khi driver không được nạp, shell có thể vẫn mở nhưng /dev/lab2 hoặc chức năng phụ thuộc sẽ không dùng được. Trong sản phẩm, cần log lỗi, tạo device node có điều kiện, tránh insmod lặp, và xác định chính sách recovery với module bắt buộc.

9. SỰ CỐ THỰC TẾ VÀ KẾT LUẬN

Sự cố / khác biệt	Biện pháp và kết quả
Lúc đầu host chưa có /proc/mtd và /dev/mtd0	Nạp nandsim, kiểm tra /proc/mtd và mtdinfo trước khi erase.
Terminal dán nhầm escape sequence	Nhập lại lệnh sạch, phân biệt output terminal và lệnh cần chạy.
Chưa có jffs2_root/etc khi tạo file	Tạo thư mục đúng, kiểm tra cấu trúc trước mkfs.jffs2.
Chip 16 MiB khác ví dụ 32 MiB	Dùng eraseblock đúng 0x4000 theo log thực tế.
rcS đã load driver trước script kiểm thử	Kiểm tra lsmod trước insmod để tránh nạp hai lần.

Kết luận: nhóm đã ghi nhận bốn nhóm kết quả: character device ARMv7 hoạt động, procfs và sysfs phản ánh được dữ liệu, JFFS2 đọc/ghi và giữ dữ liệu qua remount trên nandsim HOST, và module được nạp tự động trong chuỗi BusyBox init. Những phép đo này không thay thế kiểm thử chịu tải đồng thời, mất điện, bad-block thực hoặc triển khai OTA đầy đủ.

Tài liệu dùng để đối chiếu

[1] Đề bài LAB-02 — Embedded Linux Systems. [2] Linux Kernel Documentation: character devices, uaccess, procfs và sysfs. [3] Linux MTD/JFFS2 documentation. [4] BusyBox init. [5] Lab02_Report.pdf (tham khảo phạm vi và nội dung lý thuyết do người học cung cấp; không sử dụng ảnh hoặc số liệu của nhóm trong PDF).